NYC YELLOW TAXI PRICE PREDICTION

Group E

Team Members:

Vamshi Kumar Reddy Kovvuri (20030902) Angelina Mande (20027232)

Hanoch Surya Rao Dubbaka (20027234) Sanjavan Ghodasara (20041592)

---

1. Project Statement & Objective

In this project, we work with New York City Yellow Taxi Trip Record data. Each record is one taxi trip with pickup and dropoff time, locations, distance, passenger count, and detailed fare information. This dataset is very large (tens of millions of rows), so it is not possible to analyze or model it comfortably on a single machine without careful design.

Our main goal is to build a Spark-based system that can handle this large dataset in a scalable way. We want to clean and combine the data, explore basic patterns, and train models to predict the total fare of a trip under different data sizes and cluster settings. More specifically, our objective is to design an end-to-end pipeline that ingests raw TLC data, adds location and weather information, produces one modeling-ready table, and then uses that table to compare several regression models while also studying how performance changes when we scale up the data and scale out the cluster.

2. Data

We use the NYC Yellow Taxi Trip Record Data released by the NYC Taxi & Limousine Commission (TLC) as our main dataset. Each row is one taxi trip with fields like tpep_pickup_datetime, tpep_dropoff_datetime, PULocationID, DOLocationID, trip_distance, passenger_count, payment_type, and fare fields such as fare_amount, extra, mta_tax, tolls_amount, improvement_surcharge, congestion_surcharge, airport_fee, tip_amount, and total_amount. These fields follow TLC rules for how the taximeter calculates the fare, which helps us understand and check the total_amount column.

We also use two extra datasets to add useful context. First, we use a location dataset that maps each TLC LocationID to latitude and longitude, so we can attach approximate pickup coordinates to each trip through PULocationID. Second, we use an hourly weather dataset for New York City which has temperature, humidity, precipitation, windspeed, and general conditions (for example, clear or raining), and we match this to the pickup hour of each trip. All of these datasets are stored in Google Cloud Storage, and we read them using Spark from Colab or from a Dataproc cluster, so the data lives in the cloud and is shared across our team.

3. Data pre-processing/ETL

We built a common ETL pipeline in PySpark so that all team members work with the same cleaned and enriched data. We first read 24 Parquet files that store the taxi trips and unify differences in column names and types, then we convert pickup and dropoff times to a standard timestamp format using to_timestamp. We performed some feature engineering to add new columns like trip_duration_sec (dropoff time - pickoff time), pickup_hour, dayofweek (with 0 as Sunday to 7 as Saturday), isWeekend (1 when pickup is on Saturday or Sunday, 0 otherwise), temporal features like

year, month, day and minute (from pickup timestamp), is_rush_hour (top 30% of hours for each week day are marked as rush hours, value is 1 if it is a rush hour, 0 otherwise - Fig 3.1) and update few columns like conditions - Records with multiple weather descriptions are reduced to one primary condition using pick_primary_condition() function (Fig 3.2), airport_fee (all trips that are not from or to to airport are set to 0 - Fig 3.3), total_amount (computed the new total value by doing the summation of all the price columns except tip_amount which is not mandatory based on TLC's official fare structure). These extra columns let us capture time-based patterns such as weekend versus weekday behavior and busy rush-hour periods.

```
# Computing the rush hour by picking top 30% of hours for each week day
w_count = Window.partitionBy("dayofweek", "pickup_hour")

taxi_clean = taxi_clean.withColumn(
    "no_trips_hourly",
    F.count("*").over(w_count)
)

w_rank = Window.partitionBy("dayofweek").orderBy(F.col("no_trips_hourly").desc())

taxi_clean = taxi_clean.withColumn(
    "percentile_rank",
    F.percent_rank().over(w_rank)
)

taxi_clean = taxi_clean.withColumn(
    "is_rush_hour",
    (F.col("percentile_rank") <= 0.30).cast("int")
)
```

Fig 3.1

```
# Weather condition standardization:
# Multiple weather descriptors are reduced to one primary condition.

def pick_primary_condition(cond):
    if cond is None:
        return None
    parts = [p.strip() for p in cond.split(",")]

    if "Rain" in parts and "Snow" in parts:
        return "Sleet"

    elif "Rain" in parts:
        return "Rain"

    elif "Snow" in parts:
        return "Snow"

    else:
        return parts[0]

pick_condition_udf = F.udf(pick_primary_condition, StringType())

weather_clean = weather_clean.withColumn(
    "conditions",
    pick_condition_udf(F.col("conditions"))
)

weather_clean.select("conditions").distinct().show(truncate=False)
```

Fig 3.2

```python
# Replacing the value of airport_fee to 0 for all trips that are not from or to the airport

airport_ids = [1, 132, 138]

taxi_clean = taxi_clean.withColumn(
    "airport_fee",
    F.when(
        (F.col("PULocationID").isin(airport_ids)) |
        (F.col("DOLocationID").isin(airport_ids)),
        F.when(
            F.col("airport_fee").isNull() | (F.col("airport_fee") <= 0),
            F.lit(0)
        ).otherwise(F.col("airport_fee"))
    ).otherwise(F.lit(0))
)
```

Fig 3.3

We performed some EDA on all the datasets to better understand them and decide on the data cleaning criteria like which values can be considered as valid and which are invalid or outliers. After that, we apply filters to remove clearly wrong or unrealistic records, such as trips with distance less than or equal to zero, negative or extremely long durations, or impossible passenger counts, invalid pickup locations, etc, because these rows can distort our EDA and model training. We dropped all the unnecessary columns from all the datasets - taxi, location and weather.

Next, we enrich the trip data with location information. We join the cleaned trip DataFrame with the location table on PULocationID to attach latitude and longitude for each pickup zone, which lets us study spatial patterns and later feed coordinates into models. The location dataset is relatively small compared to the trip data, so this join is cheap but very useful for analysis.

Finally, we incorporate the weather information. We aggregate the raw weather data to hourly level and then join it on pickup_hour so that for each trip we know the temperature, humidity, precipitation, windspeed, and conditions at the time of pickup. After these steps, we obtain one big, modeling-ready table that contains cleaned trip fields, location coordinates, weather variables, and our engineered features.

We saved the final_df, which is the final dataframe after all the data cleaning to the gcp bucket which can be accessed directly to build and evaluate the models.

4. Models

We have built 4 models for the price prediction in our project - Linear Regression, Random Forest Regression, Hypertuned Random Forest Regression - number of trees are increased to 50 and depth is increased to 6, and Gradient boosted Trees Regression models.

We first converted the categorical columns, in our case the conditions from the weather dataset into numerical representations using StringIndexer to assign each unique category a numerical index. This indexed feature was then transformed using OneHotEncoder to generate a binary vector representation, ensuring the categorical data could be effectively used by machine learning models without introducing unintended ordinal relationships(Fig 4.1).

```python
from pyspark.ml.feature import StringIndexer, OneHotEncoder

# Step 1: StringIndexer for conditions
conditions_indexer = StringIndexer(inputCol="conditions", outputCol="conditionsIndex")

# Transform the data
df_enc = conditions_indexer.fit(final_df).transform(final_df)

# Step 2: OneHotEncoder for conditions
conditions_encoder = OneHotEncoder(inputCol="conditionsIndex", outputCol="conditionsVec")

# Transform the data
df_enc = conditions_encoder.fit(df_enc).transform(df_enc)

# Verify the schema
df_enc.printSchema()
```

Fig 4.1

We defined our predictor columns which are the temporal features, weather feathers and location features from the final dataframe and verifies to make sure that there are no nulls. Our target value is the total_amount which is the expected final total fare of the trip. All selected predictor variables were assembled into a single feature vector using VectorAssembler(Fig 4.2).

```python
# Defined the predictor and target columns for the models
predictor_columns = [ 'passenger_count', 'trip_distance', 'trip_duration_sec', 'pickup_hour', 'dayofweek', 'isWeekend', 'year',
                      'month', 'day', 'minute', 'is_rush_hour', 'temp', 'humidity', 'precip', 'windspeed', 'conditionsVec', 'Latitude', 'Longitude']

target_column = "total_amount"
```

+ Code + Text

```python
# Made sure there are no null values in the predictor columns
df_enc.select([
    F.count(F.when(F.col(c).isNull(), c)).alias(c)
    for c in predictor_columns
]).show(truncate=False)
```

```
+---------------+-------------+-----------------+-----------+---------+---------+----+-----+---+------+------------+----+--------+------+---------+---------------+--------+---------+
|passenger_count|trip_distance|trip_duration_sec|pickup_hour|dayofweek|isWeekend|year|month|day|minute|is_rush_hour|temp|humidity|precip|windspeed|conditionsIndex|Latitude|Longitude|
+---------------+-------------+-----------------+-----------+---------+---------+----+-----+---+------+------------+----+--------+------+---------+---------------+--------+---------+
|0              |0            |0                |0          |0        |0        |0   |0    |0  |0     |0           |0   |0       |0     |0        |0              |0       |0        |
+---------------+-------------+-----------------+-----------+---------+---------+----+-----+---+------+------------+----+--------+------+---------+---------------+--------+---------+
```

```python
# Vectorize predictors (combine all predictor columns into one feature vector)
vector_assembler = VectorAssembler(inputCols=predictor_columns, outputCol="features")
df_vectorized = vector_assembler.transform(df_enc)
```

Fig 4.2

We defined all the models by using the Spark ML(Fig 4.3). We defined our evaluation metrics - RMSE, MAE, and R^2 by using the RegressionEvaluator(Fig 4.4). We defined a function(Fig 4.5) named model_build_evaluate() - which takes estimator(model), train_data, test_data, and model_name as input. The function builds the model on the training data by fitting the estimator to the data and is applied on the test data to generate predictions which are used to evaluate the model. We also computed the training and testing time for the models.

```python
# Models

# LINEAR REGRESSION MODEL
lr = LinearRegression(featuresCol="features", labelCol=target_column)

# RANDOM FOREST REGRESSION MODEL
rf = RandomForestRegressor(featuresCol="features", labelCol=target_column)

# HYPERTUNRD RANDOM FOREST REGRESSION MODEL
hypertuned_rf = RandomForestRegressor(
    featuresCol="features",
    labelCol=target_column,
    numTrees=50,
    maxDepth=6
)

# GRADIENT - BOOSTED REGRESSION MODEL
gbt = GBTRegressor(
    featuresCol="features",
    labelCol=target_column,
    maxIter=50,
    maxDepth=4,
    seed=42
)
```

Fig 4.3

```python
# Defining evaluators - RMSE, MAE, R-square
evaluator_rmse = RegressionEvaluator(labelCol=target_column, predictionCol="prediction", metricName="rmse")
evaluator_mae = RegressionEvaluator(labelCol=target_column, predictionCol="prediction", metricName="mae")
evaluator_r2 = RegressionEvaluator(labelCol=target_column, predictionCol="prediction", metricName="r2")
```

Fig 4.4

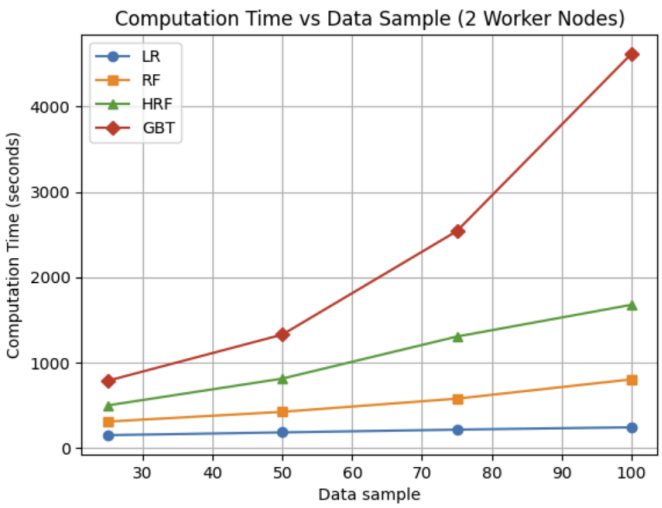
```python
# Defined a combine function to build the model and perform evaluations
# Helps to reduce repeating the same code multiple times

def model_build_evaluate(
    estimator, train_data, test_data, model_name
):
    # Training
    start_train = time.time()
    model = estimator.fit(train_data)
    training_time = time.time() - start_train

    # Testing / Prediction
    start_test = time.time()
    predictions = model.transform(test_data)
    testing_time = time.time() - start_test

    # Evaluation
    rmse = evaluator_rmse.evaluate(predictions)
    mae = evaluator_mae.evaluate(predictions)
    r2 = evaluator_r2.evaluate(predictions)
    throughput = predictions.count() / testing_time

    return {
        "model": model_name,
        "rmse": rmse,
        "mae": mae,
        "r2": r2,
        "training_time": training_time,
        "testing_time": testing_time,
        "throughput": throughput
    }
```

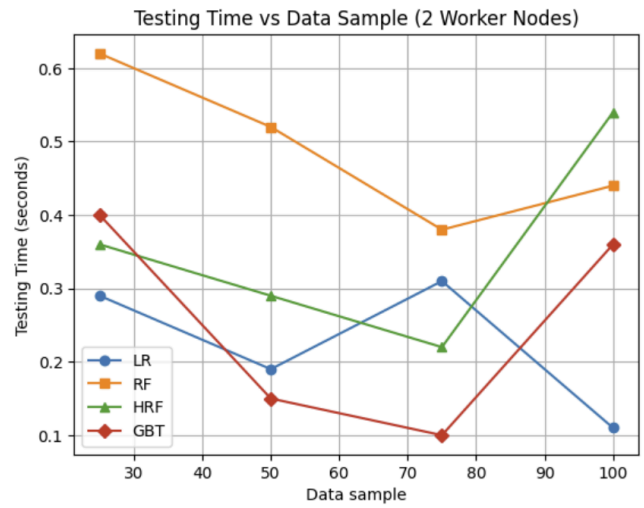
Fig 4.5

5. Scaling Up

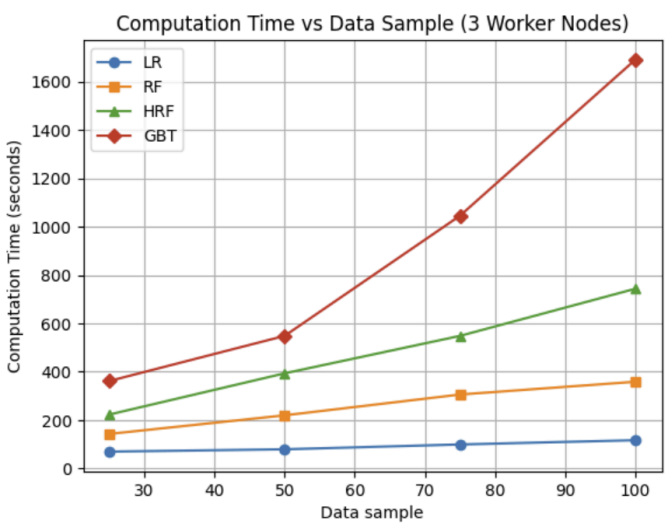
We sampled our data into 25%, 50%, 75%, and 100% which is full data. We performed all our models on all the samples of data and got the results. We splitted the training and testing as 80% and 20% respectively.

Computation and Testing time for 2 Worker Nodes:

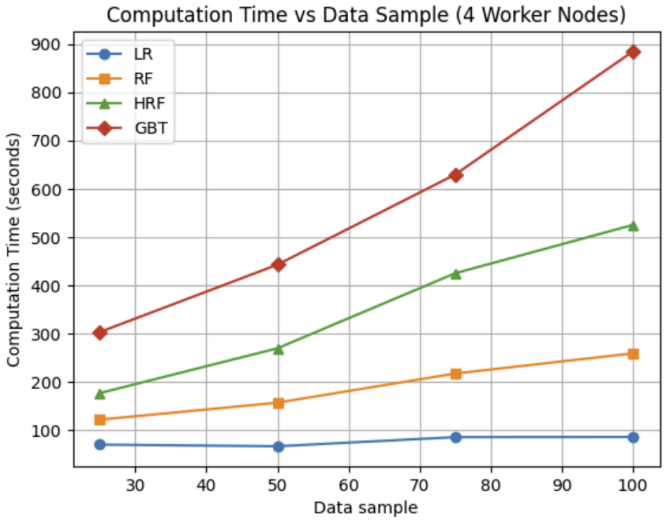




Computation and Testing time for 3 Worker Nodes:



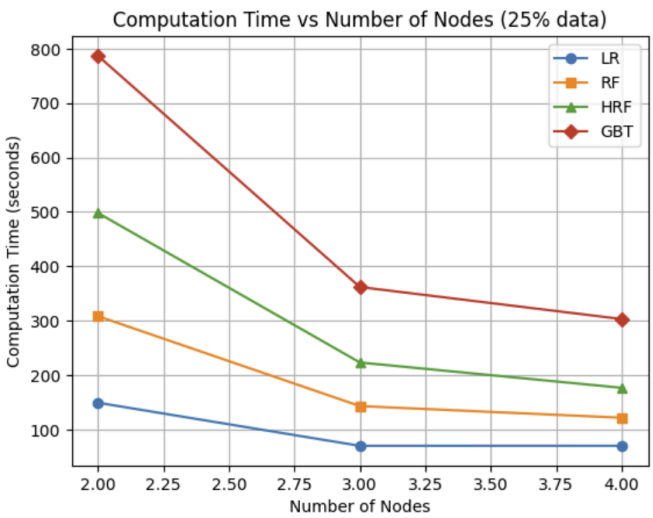
Computation and Testing time for 4 Worker Nodes:

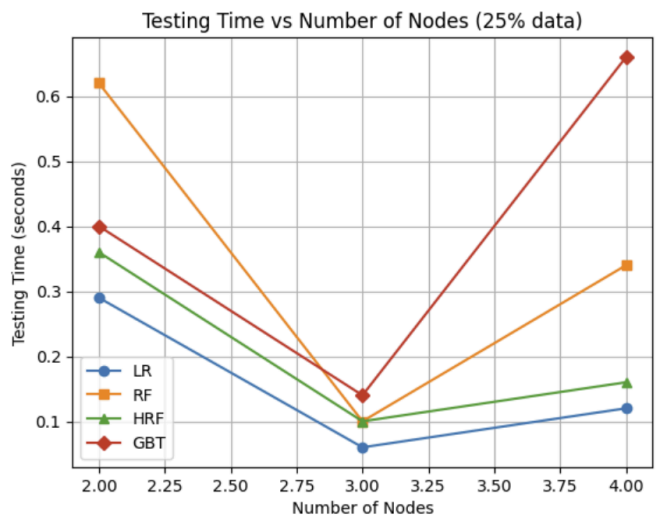


6. Scaling Out

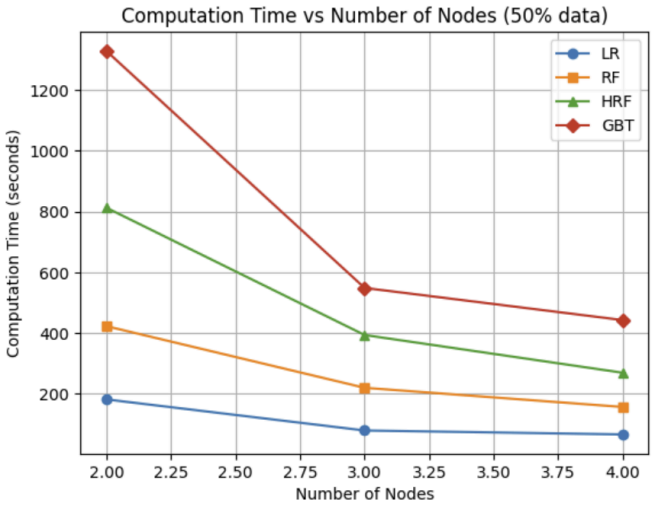
We started with 1 Master and 2 Worker nodes and increased the worker nodes gradually to 3 and 4. We ran full data on 1 Master and 4 worker nodes and achieved the goal.

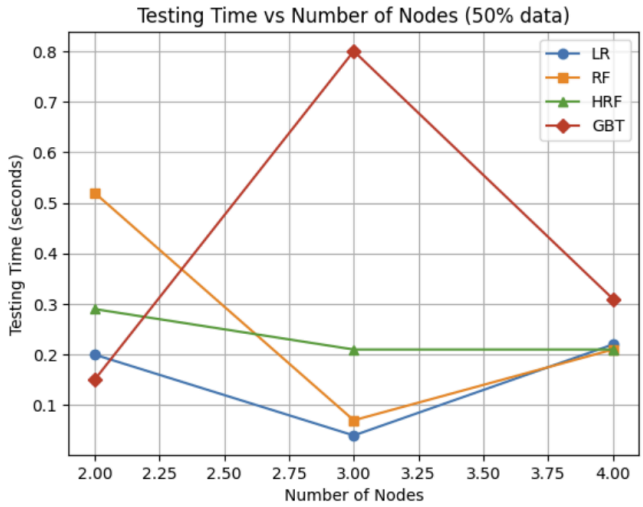
Computation and Testing time for 25% data:



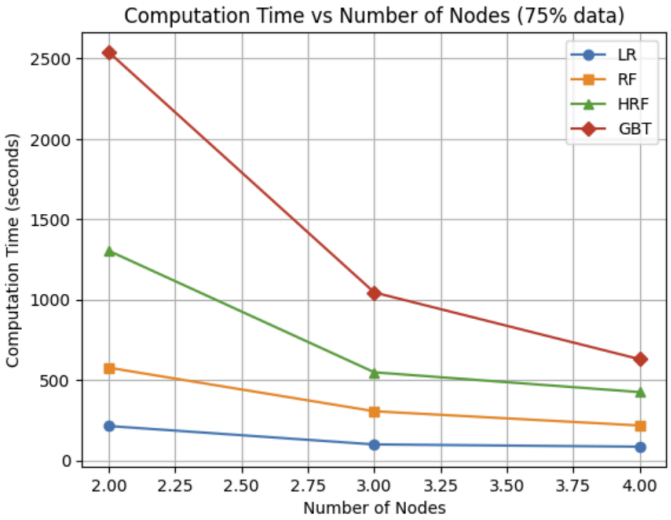


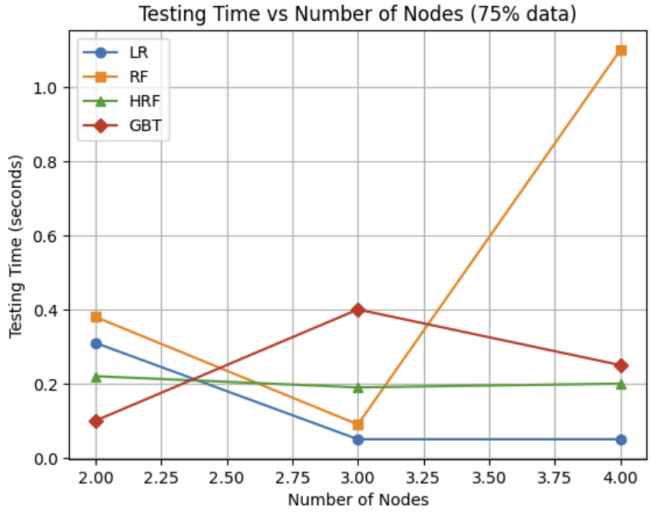
Computation and Testing time for 50% data:



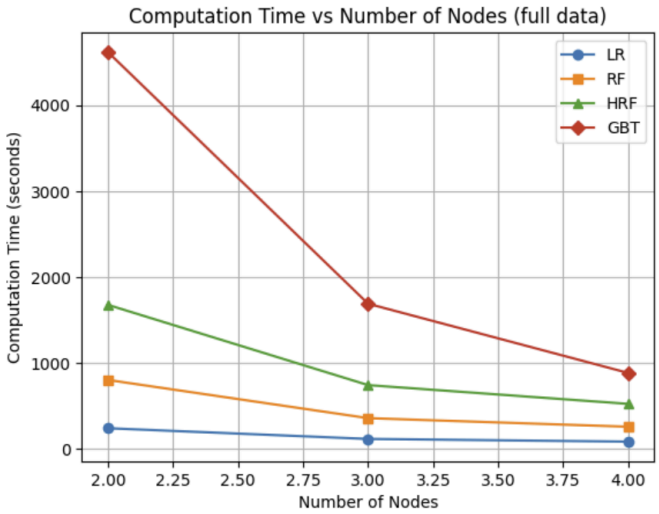


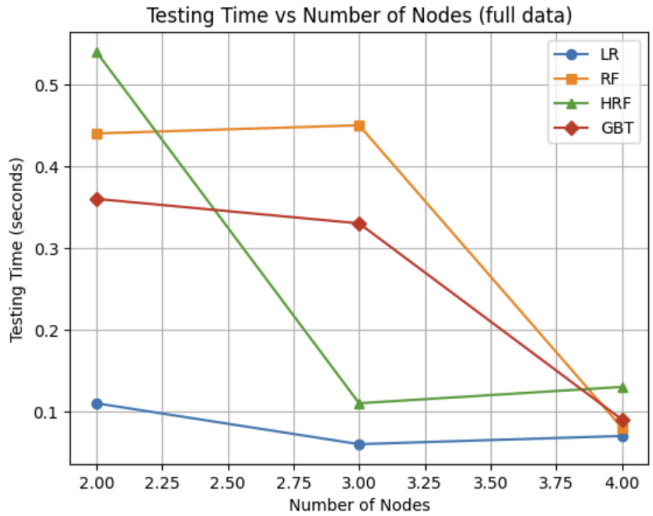
Computation and Testing time for 75% data:





Computation and Testing time for full data:





7. Conclusion

In this project, we built an end-to-end Spark-based system on Google Cloud to process NYC Yellow Taxi Trip data at scale, starting from raw TLC trip records and applying a shared PySpark ETL pipeline to clean the data, validate fare components, and join trips with location and weather datasets stored in cloud storage, which gave us one modeling-ready table that the whole team used for EDA and modeling. On top of this table, we trained and compared multiple regression models (Linear Regression, Random Forest, tuned Random Forest, and Gradient Boosted Trees) to predict total_amount, and we evaluated them with RMSE, MAE, and R^2 while also measuring training and testing time under different data samples and Dataproc cluster configurations, which showed that tree-based models can reach strong accuracy and benefit from distributed computing, and that scaling out mainly improves speed rather than changing model quality.

8. Future Scope

In the future, we plan to extend this work by integrating more external datasets that can influence taxi demand and fares, such as public events, concerts, sports games, city festivals, and official holidays, because these factors are known to change how often people use taxis and when demand spikes in a city. We also aim to move beyond notebooks and build a simple web application and a public API on top of our trained models, so that end users and developers can send inputs like pickup time, pickup location, trip distance, and possibly basic weather or event information, and receive real-time fare predictions using our Spark-based models in a practical, real-world setting.